

Álgebra en FEC

Trabajo Práctico 1

1 Campos de Galois

Diseñar una clase en Python que represente un campo de Galois $GF(2^m)$. El constructor debe recibir el orden m del campo y el polinomio primitivo $P(x)$, representado como un entero de m bits (sin el término x^m , que queda implícito en la reducción). Los elementos del campo se representan como enteros en el rango $[0, 2^m - 1]$.

La clase debe implementar las siguientes operaciones sobre elementos del campo:

- Suma
- Producto
- Inverso multiplicativo (indicando el comportamiento para el elemento 0)
- División (indicando el comportamiento para divisor 0)
- Potencia A^n , para $n \geq 0$

Los elementos del campo se representan como enteros en el rango $[0, 2^m - 1]$, resultado de interpretar la tupla de m coeficientes binarios $(b_{m-1}, \dots, b_1, b_0)$ como un número en base 2.

2 Polinomios sobre un Campo de Galois

Utilizando la clase desarrollada previamente, diseñar una clase `GFPoly` que represente un polinomio con coeficientes en $GF(2^m)$.

El constructor debe recibir una instancia de la clase anterior y una lista (o tupla) de coeficientes en orden decreciente de grado, es decir, $[a_n, a_{n-1}, \dots, a_1, a_0]$ representa el polinomio

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0.$$

Debe validarse que cada coeficiente sea un elemento válido del campo, y que la representación interna no contenga ceros a la izquierda salvo en el caso del polinomio nulo.

La clase debe implementar las siguientes operaciones, todas ellas realizadas sobre el campo $GF(2^m)$ subyacente:

- **Suma** de dos polinomios
- **Producto** de dos polinomios.
- **División entera** de dos polinomios, obteniendo por separado el **cociente** y el **resto**.
- **Escalado**: multiplicar todos los coeficientes de un polinomio por un escalar del campo.
- **Evaluación** del polinomio en un punto $x \in GF(2^m)$.
- **Construcción a partir de raíces**: dado un conjunto de raíces $\{r_1, \dots, r_k\} \subset GF(2^m)$, construir el polinomio $\prod_{i=1}^k (x - r_i)$.

Se recomienda sobrecargar los operadores de Python correspondientes $(+, *, //, \%, ==)$ para que las operaciones puedan escribirse de forma natural sobre objetos de la clase.